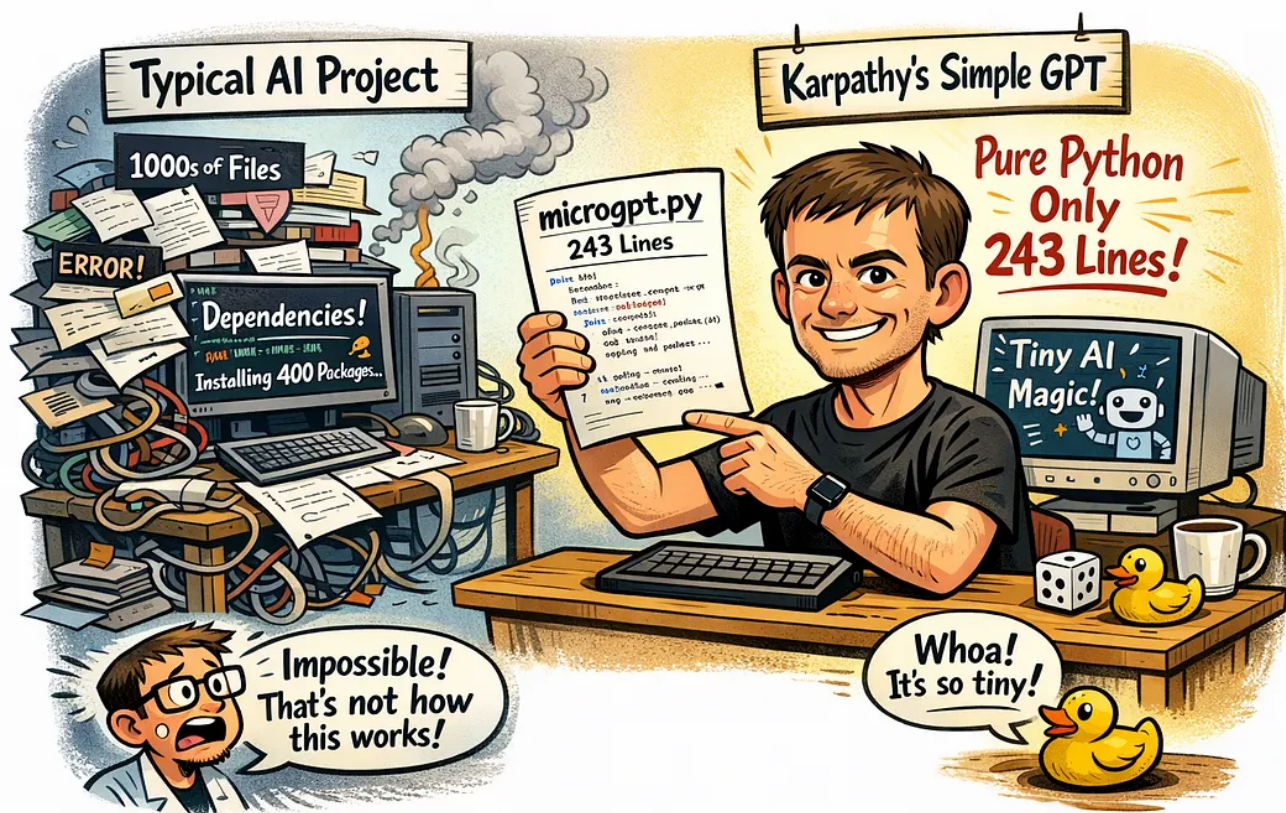




I've read many transformer implementations during my PhD. Dense codebases. Thousands of files. Dependencies stacked on top of dependencies. You open a repo, run `pip install -r requirements.txt`, and watch 400 packages download before you can even see your model train (than errors, dependency issues ... etc.).

Then on February 11, 2026, Andrej Karpathy dropped a single Python file that trains and runs a GPT from scratch. 243 lines. Zero dependencies.

If you cant read the article further than please click [here](#)

His only imports? `os`, `math`, `random`, and `argparse`. That's it. That's the entire LLM. He called it an "art project." I call it the best AI education that exists on the internet right now. Let me walk you through every piece of this code like I'm explaining it to a friend over coffee.

First, What Is This Thing Actually Doing?

Before we touch the code, let's be clear about what microGPT does. It downloads a list of baby names. It learns the patterns in those names. Then it generates new, fake names that *sound* real but never existed.

That's it. Same concept as ChatGPT, just tiny. ChatGPT learned from the entire internet. This model learns from a text file of names. But the *algorithm* is identical. The attention mechanism, the training loop, the way it predicts the next token: it's the same math running inside GPT-4. Just at a micro scale.

Think of it like this: a toy car and a Tesla both have engines, wheels, steering, and brakes. The toy car won't win any races, but if you want to understand how a car works, the toy car is perfect.

Part 1: The Tokenizer (Lines You'll Forget Are Important)

```
chars = ['<BOS>', '<EOS>'] + sorted(list(set(''.join(docs))))
vocab_size = len(chars)
stoi = { ch:i for i, ch in enumerate(chars) }
itos = { i:ch for i, ch in enumerate(chars) }
```

Computers don't understand letters. They understand numbers.

So the very first job is to convert every character into a number. The letter `a` becomes `2`, `b` becomes `3`, and so on. There are also two special tokens:

<BOS> (Beginning Of Sequence, "*Hey model, a new name is starting!*") and
<EOS> (End Of Sequence "*This name is done now!*").

`stoi` converts strings to integers. `itos` converts integers back to strings.
That's your entire tokenizer.

ChatGPT uses a much fancier tokenizer called BPE (Byte Pair Encoding) that groups common letter combinations into single tokens. But the *idea* is identical: text in, numbers out.

Part 2: The Autograd Engine (The Heart of Everything)

This is where it gets beautiful.

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0
        self._backward = lambda: None
        self._prev = set(_children)
```

Karpathy builds a mini version of PyTorch's autograd engine in about 40 lines.

Let me explain why this matters.

When a neural network makes a prediction, it's usually wrong at first. We need to figure out: *which numbers (weights) should we adjust, and by how much, to make it less wrong?*

That's what **backpropagation** does. And backpropagation needs **gradients** — basically, it needs to know “*if I wiggle this weight a tiny bit, how much does the final error change?*”

The `Value` class wraps every single number in the network and tracks:

- `data`: the actual number
- `grad`: how much the final loss changes when you change this number
- `_backward`: instructions for computing that gradient
- `_prev`: which other `Values` were used to create this one

Every time you do math (add, multiply, power, etc.), the `Value` class quietly builds a graph of operations behind the scenes. When training is done for one example, you call `.backward()` and the gradients flow back through this graph automatically.

Here's addition:

```
def __add__(self, other):
    out = Value(self.data + other.data, (self, other), '+')
    def _backward():
        self.grad += out.grad
        other.grad += out.grad
    out._backward = _backward
    return out
```

If $c = a + b$, and someone tells you "*c's gradient is 5*," then both a 's and b 's gradient is also 5. Because if you increase a by 1, c also increases by 1. That's the chain rule from calculus, implemented in 4 lines.

Multiplication is similar but uses the “*swap*” rule:

```
def __mul__(self, other):
    out = Value(self.data * other.data, (self, other), '*')
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

If $c = a * b$, then a 's gradient is $b * (\text{gradient of } c)$ and b 's gradient is $a * (\text{gradient of } c)$. That's it.

The full backward pass uses **topological sort**: it just makes sure you compute gradients in the right order (outputs first, inputs last):

```
def backward(self):
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)
    self.grad = 1
    for v in reversed(topo):
        v._backward()
```

This is literally the same algorithm that runs inside PyTorch when you call `loss.backward()`. Karpathy just wrote it from scratch using Python lists and recursion.

Part 3: Model Parameters (The “Brain” of the GPT)

```
matrix = lambda nout, nin, std=0.02: [[Value(random.gauss(0, std)) for _ in range(nout)] for _ in range(nin)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd)}
```

Every neural network is just a collection of numbers (called **weights** or **parameters**). Before training, these are random. After training, they encode learned patterns.

Here's what each weight matrix does:

- **wte** (Word Token Embedding): Converts each token ID into a vector of 16 numbers. Think of it as giving each character a "personality": a position in a 16-dimensional space where similar characters end up close together.
- **wpe** (Word Position Embedding): Tells the model *where* in the sequence each character sits. "a" at position 1 should be treated differently than "a" at position 5.

For each transformer layer, there are:

- **attn_wq**, **attn_wk**, **attn_wv**: The Query, Key, and Value matrices for attention (more on this below)
- **attn_wo**: The output projection of attention
- **mlp_fc1**, **mlp_fc2**: The feedforward network that processes the attended information

Default configuration: 16 embedding dimensions, 4 attention heads, 1 layer, sequence length of 8. This gives you roughly 4,000 parameters. GPT-4 has over a trillion. Same architecture, wildly different scale.

Part 4: The GPT Architecture (Where the Magic Happens)

This is the core. Let me break down the `gpt()` function piece by piece.

Step 1: Embedding

```
tok_emb = state_dict['wte'][token_id]
pos_emb = state_dict['wpe'][pos_id % block_size]
x = [t + p for t, p in zip(tok_emb, pos_emb)]
```

Take the character's embedding. Take the position's embedding. Add them together. Now the model knows both *what* the character is and *where* it sits.

Step 2: RMSNorm

```
def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]
```

Before each major computation, we normalize the values so they don't explode or shrink to zero. RMSNorm is a simpler cousin of LayerNorm (which original GPT-2 uses). It divides each value by the root-mean-square of

all values.

Think of it as keeping everyone's volume at a reasonable level before a group discussion.

Step 3: Attention (The Star of the Show)

This is the part everyone talks about. “Attention is All You Need”: the famous 2017 paper. Here it is, naked, in pure Python:

```
q = linear(x, state_dict[f'layer{li}.attn_wq'])
k = linear(x, state_dict[f'layer{li}.attn_wk'])
val = linear(x, state_dict[f'layer{li}.attn_wv'])
```

For the current token, compute three things:

- **Query (Q):** “*What am I looking for?*”
- **Key (K):** “*What do I contain?*”
- **Value (V):** “*What information do I carry?*”

Then for each attention head:

```
attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5
                for t in range(len(k_h))]
attn_weights = softmax(attn_logits)
head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h)))
            for j in range(head_dim)]
```

Here's what's happening in plain English:

1. The current token's Query asks: “Which previous tokens are relevant to me?”
2. It computes a dot product with every previous token's Key (like a compatibility score)
3. These scores are passed through softmax (making them add up to 1, like probabilities)
4. The Value vectors of all tokens are then mixed together, weighted by these scores

So if the model is generating the name “**Micha**” and it's about to predict the next letter, the attention mechanism lets the current position “look back” at M, i, c, h, and a, and decide which ones matter most for predicting what comes next.

The division by `head_dim**0.5` is called **scaled attention**: it prevents the dot products from getting too large, which would make softmax output extreme probabilities (basically all 0s and one 1).

Multi-head attention means we run this process 4 times in parallel (with 4 different Q, K, V projections), each looking at a different 4-dimensional slice of the 16-dimensional embedding. One head might learn to focus on vowels. Another might focus on recent characters. They all bring different perspectives.

Step 4: Residual Connection

```
x = [a + b for a, b in zip(x, x_residual)]
```

After attention, we add back the original input. This is called a **residual connection** (or skip connection).

Why? Because it creates a “**highway**” for gradients during training. Without it, deep networks become nearly impossible to train. It’s like saying: “Whatever you learned from attention, just add it to what you already knew.”

Step 5: MLP (Feed-Forward Network)

```
x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
x = [xi.relu() ** 2 for xi in x]
x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
```

After attention decides which tokens to pay attention to, the MLP processes that information.

The MLP expands the 16-dimensional vector to 64 dimensions ($4 * n_embd$), applies a non-linearity (squared ReLU), and squishes it back to 16.

Why expand and compress? Think of it like brainstorming. You first generate lots of ideas (expand to 64), filter out the bad ones (ReLU kills negatives, squaring amplifies strong signals), then summarize (compress back to 16).

Squared ReLU instead of GELU is one of Karpathy’s simplifications. Regular ReLU says: “if negative, make it 0.” Squared ReLU says: “if negative, make it 0; if positive, square it.” This gives stronger non-linearity.

Step 6: Project to Vocabulary

```
logits = linear(x, state_dict['wte'])
```

Finally, the model uses the same embedding matrix (w_{te}) to project back to vocabulary size. This is called **weight tying**, the same weights that convert token IDs into embeddings are reused to convert embeddings back into predictions.

The output is a score for each possible next character. Higher score = model thinks this character is more likely to come next.

Part 5: The Training Loop (How It Learns)

```
for step in range(args.num_steps):
    doc = docs[step % len(docs)]
    tokens = [BOS] + [stoi[ch] for ch in doc] + [EOS]
    tokens = tokens[:block_size]
```

For each step, grab one name from the dataset. Convert it to token IDs. Add BOS at the start and EOS at the end.

```
for pos_id in range(len(tokens) - 1):
    logits = gpt(tokens[pos_id], pos_id, keys, values)
    probs = softmax(logits)
    loss = -probs[tokens[pos_id + 1]].log()
```

For each position, the model sees the current token and predicts the next one. The loss is **negative log likelihood**: if the model assigns high probability to the correct next character, the loss is low. If it assigns low probability, the loss is high.

Then `loss.backward()` computes gradients for every parameter, and the **Adam optimizer** updates them:

```
m[i] = beta1 * m[i] + (1 - beta1) * p.grad
v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
m_hat = m[i] / (1 - beta1 ** (step + 1))
v_hat = v[i] / (1 - beta2 ** (step + 1))
p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
```

Adam is like gradient descent with memory. Instead of just following the current gradient, it tracks:

- m : the average direction of recent gradients (momentum: like a ball rolling downhill)
- v : the average magnitude of recent gradients (helps normalize: big gradients don't dominate)

This is the same optimizer used to train GPT-4, DALL-E, and every major model today.

Part 6: Inference (Generating New Names)

```

for sample_idx in range(5):
    token_id = BOS
    generated = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        token_id = random.choices(range(vocab_size), weights=[p.data for p in pr
    if token_id == EOS:
        break
    generated.append(itos[token_id])

```

Start with BOS. Ask the model “what comes next?” Sample from the probability distribution. Feed that prediction back in. Repeat until the model outputs EOS or we run out of space.

This is **autoregressive generation**: the same thing ChatGPT does every time you send it a message. One token at a time, each conditioned on everything before it.

The Big Picture: Why This Matters

Here’s the thing that blew my mind.

Every line of code beyond these 243 lines: the thousands of files in PyTorch, the CUDA kernels, the distributed training frameworks, the fancy tokenizers, **all of that is just optimization**. Making it faster. Making it bigger. Making it run across thousands of GPUs.

But the *algorithm*? The actual intelligence? It’s right here.

As Karpathy put it: “*This is the full algorithmic content of what is needed.*”

Everything else is just for efficiency. I cannot simplify this any further.”

This is Karpathy’s 6-year compression journey:

- **2020:** micrograd (autograd engine)
- **2020:** minGPT (PyTorch GPT)
- **2023:** nanoGPT (production-grade training)
- **2024:** llm.c (raw C/CUDA, no frameworks)
- **2026:** microGPT (the algorithm and nothing else)

Each step peeled away a layer of abstraction. This final version removed all of them.

The industry is spending \$400 billion on AI infrastructure this year. But the core algorithm that powers all of it? It fits in a file smaller than most README documents.

If you’re trying to understand how LLMs work, don’t start with the PyTorch documentation. Start here.

If you found this useful, follow me on [Towards Deep Learning](#) where I break down the latest AI research into plain English. And if you want to run this yourself, grab the code from [Karpathy’s GitHub Gist](#) and just type `python microgpt.py`. No installs needed.

Chat Gpt Ai

Python

Artificial Intelligence

Deep Learning

Machine Learning